

SONiX 32-Bit Cortex-M0 Micro-Controller

GPIO Example Code

SONiX reserves the right to make change without further notice to any products herein to improve reliability, function or design. SONiX does not assume any liability arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights nor the rights of others. SONiX products are not designed, intended, or authorized for use as components in systems intended, for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the SONiX product could create a situation where personal injury or death may occur. Should Buyer purchase or use SONiX products for any such unintended or unauthorized application. Buyer shall indemnify and hold SONiX and its officers, employees, subsidiaries, affiliates and distributors harmless against all claims, cost, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use even if such claim alleges that SONiX was negligent regarding the design or manufacture of the part.



AMENDMENT HISTORY

Version	Date	Description
1.0	2024/11/29	1. First version.

Table of Content

AMENDMENT HISTORY	2
1 OVERVIEW.....	4
2 FEATURE	4
3 HW	4
4 EXAMPLE CODE.....	5
5 RESULT	7

1 OVERVIEW

This document provides our customers the C program examples of GPIO.

2 FEATURE

- I/O toggle
- There are 5 test function of IO setting interrupt trigger (rising edge trigger, falling edge trigger, both edge trigger, high level trigger, and low level trigger)
- Use ARM-compiler V6.12

3 HW

- The starter kit board of vendor desired MCU.

4 EXAMPLE CODE

1. We use GPIO_Init function to set I/O input, output, trigger type, and explain as follow:

(1) Set P2.0 to be Input Pull-down, rising edge trigger and enable IE interrupt.

*** Note: For MCU which does not support Input Pull-down, In order to demo rising edge trigger interrupt, SHALL set P2.0 to input floating and add an external 50K Resistor to VSS.**

```

45 //P2.0 as Input Pull-down
46 GPIO_Mode      (GPIO_PORT2, GPIO_PIN0, GPIO_MODE_INPUT);
47 GPIO_Config    (GPIO_PORT2, GPIO_PIN0, GPIO_CFG_PULL_DOWN);
48 //P2.0 as rising edge
49 GPIO_P2Trigger(GPIO_PIN0, GPIO_IS_EDGE, GPIO_IBS_EDGE_TRIGGER, GPIO_IEV_RISING_EDGE);
50 GPIO_Interrupt(GPIO_PORT2, GPIO_PIN0, GPIO_IE_EN);

```

(2) Set P2.1 to Input Pull-up, falling edge trigger and enable IE interrupt.

```

52 //P2.1 as Input Pull-up
53 GPIO_Mode      (GPIO_PORT2, GPIO_PIN1, GPIO_MODE_INPUT);
54 GPIO_Config    (GPIO_PORT2, GPIO_PIN1, GPIO_CFG_PULL_UP);
55 //P2.1 as falling edge
56 GPIO_P2Trigger(GPIO_PIN1, GPIO_IS_EDGE, GPIO_IBS_EDGE_TRIGGER, GPIO_IEV_FALLING_EDGE);
57 GPIO_Interrupt(GPIO_PORT2, GPIO_PIN1, GPIO_IE_EN);

```

(3) Set P2.2 to Input Repeater-mode, both edge trigger and enable IE interrupt.

*** Note: Only SN32F700/100~~/760/240~~ support Input Repeater-mode.**

```

59 //P2.2 as Input Repeater-mode
60 GPIO_Mode      (GPIO_PORT2, GPIO_PIN2, GPIO_MODE_INPUT);
61 GPIO_Config    (GPIO_PORT2, GPIO_PIN2, GPIO_CFG_REPEATER_MODE);
62 //P2.2 as both edge
63 GPIO_P2Trigger(GPIO_PIN2, GPIO_IS_EDGE, GPIO_IBS_BOTH_EDGE_TRIGGER, GPIO_IEV_RISING_EDGE);
64 GPIO_Interrupt(GPIO_PORT2, GPIO_PIN2, GPIO_IE_EN);

```

(4) Set P2.3 to Input Pull-down, high level edge trigger and enable IE interrupt.

*** Note: For MCU which does not support Input Pull-down, In order to demo rising edge trigger interrupt, SHALL set P2.0 to input floating and add an external 50K Resistor to VSS.**

```

66 //P2.3 as Input Pull-down
67 GPIO_Mode      (GPIO_PORT2, GPIO_PIN3, GPIO_MODE_INPUT);
68 GPIO_Config     (GPIO_PORT2, GPIO_PIN3, GPIO_CFG_PULL_DOWN);
69 //P2.3 as high level
70 GPIO_P2Trigger(GPIO_PIN3, GPIO_IS_EVENT, GPIO_IBS_EDGE_TRIGGER, GPIO_IEV_RISING_EDGE);
71 GPIO_Interrupt(GPIO_PORT2, GPIO_PIN3, GPIO_IE_EN);

```

(5) Set P2.4 to Input Pull-up, low level edge trigger and enable IE interrupt.

```

73 //P2.4 as Input Pullup
74 GPIO_Mode      (GPIO_PORT2, GPIO_PIN4, GPIO_MODE_INPUT);
75 GPIO_Config     (GPIO_PORT2, GPIO_PIN4, GPIO_CFG_PULL_UP);
76 //P2.4 as low level trigger
77 GPIO_P2Trigger(GPIO_PIN4, GPIO_IS_EVENT, GPIO_IBS_EDGE_TRIGGER, GPIO_IEV_FALLING_EDGE);
78 GPIO_Interrupt(GPIO_PORT2, GPIO_PIN4, GPIO_IE_EN);

```

(6) Set P2.5 to Output low.

```

80 //P2.5 as Output Low
81 GPIO_Mode      (GPIO_PORT2, GPIO_PIN5, GPIO_MODE_OUTPUT);
82 GPIO_Clr       (GPIO_PORT2, GPIO_PIN5);

```

2. Call GPIO_Init();

```

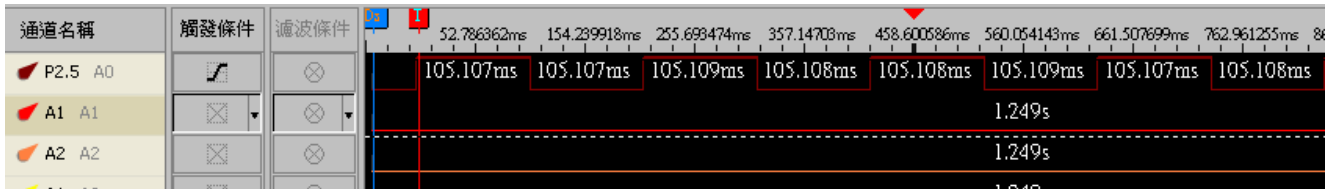
int  main (void)
{
    //init GPIO
    GPIO_Init();

    .
    .
    User code
    .
    .
}

```

5 RESULT

1. P2.5 toggles every 100ms.



2. Because P2.0 is setting as rising edge trigger, if P2.0 is triggered by rising edge, the program will enter P2_IRQHandler and stop at the break point.

```

678 void P2_IRQHandler(void)
679 {
680     if (GPIO_IntStatus(GPIO_PORT2,GPIO_PIN0)==1)
681     {
682         GPIO_IntClr(GPIO_PORT2,GPIO_PIN0);
683     }

```

3. Because P2.1 is setting as rising edge trigger, if P2.0 is triggered by falling edge, the program will enter P2_IRQHandler and stop at the break point.

```

678 void P2_IRQHandler(void)
679 {
680     if (GPIO_IntStatus(GPIO_PORT2,GPIO_PIN0)==1)
681     {
682         GPIO_IntClr(GPIO_PORT2,GPIO_PIN0);
683     }
684     else if(GPIO_IntStatus(GPIO_PORT2,GPIO_PIN1)==1)
685     {
686         GPIO_IntClr(GPIO_PORT2,GPIO_PIN1);
687     }

```

4. Because P2.2 is setting as both edge trigger, if P2.2 is triggered by either rising edge or falling edge, and the program will enter P2_IRQHandler and stop at the break point.

```

678 void P2_IRQHandler(void)
679 {
680     if (GPIO_IntStatus(GPIO_PORT2,GPIO_PIN0)==1)
681     {
682         GPIO_IntClr(GPIO_PORT2,GPIO_PIN0);
683     }
684     else if(GPIO_IntStatus(GPIO_PORT2,GPIO_PIN1)==1)
685     {
686         GPIO_IntClr(GPIO_PORT2,GPIO_PIN1);
687     }
688     else if(GPIO_IntStatus(GPIO_PORT2,GPIO_PIN2)==1)
689     {
690         GPIO_IntClr(GPIO_PORT2,GPIO_PIN2);
691     }

```

5. Because P2.3 is setting as high level trigger, if P2.3 is triggered by high level, the program will enter P2_IRQHandler and stop at the break point.

```
678 void P2_IRQHandler(void)
679 {
680     if (GPIO_IntStatus(GPIO_PORT2,GPIO_PIN0)==1)
681     {
682         GPIO_IntClr(GPIO_PORT2,GPIO_PIN0);
683     }
684     else if(GPIO_IntStatus(GPIO_PORT2,GPIO_PIN1)==1)
685     {
686         GPIO_IntClr(GPIO_PORT2,GPIO_PIN1);
687     }
688     else if(GPIO_IntStatus(GPIO_PORT2,GPIO_PIN2)==1)
689     {
690         GPIO_IntClr(GPIO_PORT2,GPIO_PIN2);
691     }
692     else if(GPIO_IntStatus(GPIO_PORT2,GPIO_PIN3)==1)
693     {
694         GPIO_IntClr(GPIO_PORT2,GPIO_PIN3);
695     }
```

6. Because P2.4 is setting as low level trigger, if P2.4 is triggered by low level, the program will enter P2_IRQHandler and stop at the break point.

```
678 void P2_IRQHandler(void)
679 {
680     if (GPIO_IntStatus(GPIO_PORT2,GPIO_PIN0)==1)
681     {
682         GPIO_IntClr(GPIO_PORT2,GPIO_PIN0);
683     }
684     else if(GPIO_IntStatus(GPIO_PORT2,GPIO_PIN1)==1)
685     {
686         GPIO_IntClr(GPIO_PORT2,GPIO_PIN1);
687     }
688     else if(GPIO_IntStatus(GPIO_PORT2,GPIO_PIN2)==1)
689     {
690         GPIO_IntClr(GPIO_PORT2,GPIO_PIN2);
691     }
692     else if(GPIO_IntStatus(GPIO_PORT2,GPIO_PIN3)==1)
693     {
694         GPIO_IntClr(GPIO_PORT2,GPIO_PIN3);
695     }
696     else if(GPIO_IntStatus(GPIO_PORT2,GPIO_PIN4)==1)
697     {
698         GPIO_IntClr(GPIO_PORT2,GPIO_PIN4);
699     }
```


SONiX reserves the right to make change without further notice to any products herein to improve reliability, function or design. SONiX does not assume any liability arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights nor the rights of others. SONiX products are not designed, intended, or authorized for use as components in systems intended, for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the SONiX product could create a situation where personal injury or death may occur. Should Buyer purchase or use SONiX products for any such unintended or unauthorized application. Buyer shall indemnify and hold SONiX and its officers, employees, subsidiaries, affiliates and distributors harmless against all claims, cost, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use even if such claim alleges that SONiX was negligent regarding the design or manufacture of the part.

Main Office:

Address: 10F-1, NO. 36, Taiyuan Street, Chupei City, Hsinchu, Taiwan R.O.C.
Tel: 886-3-5600 888
Fax: 886-3-5600 889

Taipei Office:

Address: 15F-2, NO. 171, Song Ted Road, Taipei, Taiwan R.O.C.
Tel: 886-2-2759 1980
Fax: 886-2-2759 8180

Hong Kong Office:

Unit No.705, Level 7 Tower 1, Grand Central Plaza 138 Shatin Rural Committee Road, Shatin, New Territories, Hong Kong.
Tel: 852-2723-8086
Fax: 852-2723-9179

Technical Support by Email:

Sn8fae@sonix.com.tw